

NATIONAL UNIVERSITY OF SINGAPORE

CS5425/CS4225 – Big Data Systems for Data Science
Semester 1 AY2025/2026 (Sample Answers)

Time Allowed: 2 Hours

INSTRUCTIONS TO STUDENTS:

1. Please write your Student Number only. Do not write your name.
2. This assessment paper contains **SIX(6)** questions and comprises **NINETEEN(19)** printed pages, including this page.
3. Students are required to answer **ALL** the questions.
4. Write your answers within the space provided. Answers written on other parts of the answer script will not be graded unless you specify explicitly.
5. This is an **OPEN BOOK** examination, allowing any physical materials and electronic calculator but no other electronic devices allowed.
6. **You should finish the test strictly on your own.** Do not discuss/share/copy with other students. **We have zero tolerance on plagiarism and cheating.**
7. Please circle your class below.

Class: CS5425 CS4225

Student Number: _____

This portion is for examiner's use only

Question	Marks	Remarks
Q1 [12 marks]		
Q2 [13 marks]		
Q3 [6 marks]		
Q4 [6 marks]		
Q5 [6 marks]		
Q6 [7 marks]		
Total [50 marks]		

QUESTION 1: Indicate whether the following statements are true or false. Briefly justify your answers. Each question is worth two marks. [12 marks]

- 1) Define “near-duplicate pairs” as documents with Jaccard similarity of 0.9 and above. If two documents have the same MinHash signature, they are guaranteed to be near-duplicate pairs.

Your Answer (True / False): False (1 mark)

Justification:

MinHash only gives us a fast approximation of the near-duplicate pairs. Two documents sharing only a small number of shingles can have the same MinHash value by chance. (1 mark)

- 2) In any MapReduce program, directly re-using the reduce function as a combine function (i.e., defining a combine function which is identical to the reduce function) does not affect the final output of the program.

Your Answer (True / False): False (1 mark)

Justification:

For example, if the reduce function is not commutative or associative (such as the “average” function), computing it in the combine function will change the final results. (1 mark)

- 3) In a distributed database, shortly after performing an update, we read the same data from multiple replicas. All of them returned the updated value. This means the system is *not* eventually consistent.

Your Answer (True / False): False (1 mark)

Justification:

Eventual consistency does not forbid the system from behaving consistently; it only means we can only guarantee that the system will be consistent after waiting for a sufficient period of time. (1 mark)

- 4) The DataFrame API in Spark offers immutable data structures, similar to RDDs. This means modification on an existing DataFrame results in a new DataFrame.

Your Answer (True / False): True (1 mark)

Justification:

Spark DataFrames are immutable; all transformation operations on a DataFrame produce a new DataFrame without modifying the original. This design pattern is helpful for fault tolerance. (1 mark)

- 5) In Flink if the watermarks records are not properly injected, the event window can be remained open indefinitely.

Your Answer (True / False): True (1 mark)

Justification: In Flink, watermarks signal the most likely current event time. So based on watermarks records, Flink can determine when to close the event time window and stop waiting for the late events. Therefore, if watermarks are not properly injected, the event window can be remained open indefinitely. (1 mark)

- 6) Pregel's computation model guarantees that all vertices will stop execute once no more messages are being sent and all vertices have voted to halt.

Your Answer (True / False): True (1 mark)

Justification: Pregel vertices vote to halt when their computations are done, and the system only terminates when all vertices have voted to halt and no further messages are in transit. (1 mark)

QUESTION 2: MapReduce [13 marks]

Over a set of text documents, our goal is to compute, for each word, **the word(s) that most frequently come immediately after it**. For example, for the word “*kent*”, the word that most frequently comes after it may be “*ridge*”.

Input: Our input contains multiple documents, each represented as a single line. Each map call receives one key-value pair of the form $\langle \text{document_id}, \text{line} \rangle$, where *document_id* is an integer starting from 1, 2, ..., and *line* is a **space-separated string** containing the text of the line. You can assume the text in *line* has been “cleaned”; i.e. all words have been converted to lowercase, no characters exist other than letters and spaces, and there are no duplicate spaces.

Output: For each distinct word that appeared in the document, your reduce function should emit a key-value pair of the form $\langle \text{word}, \text{list_of_most_frequent_next_words} \rangle$. You may output the list in any order. If multiple words have the same maximum frequency, include all of them in the list. If a word only appears at the end of the document, do not emit it as a key. For example, given the 2 documents:

1: *the cat*
2: *cat in the hat*

The output should consist of the following tuples:

$\langle \text{the}, [\text{cat}, \text{hat}] \rangle, \langle \text{cat}, [\text{in}] \rangle, \langle \text{in}, [\text{the}] \rangle$

Explanation: After “*the*”, both “*cat*” and “*hat*” appear once each.
After “*cat*”, only “*in*” appears once, in document 2. (In document 1, “*cat*” appears at the end of the document, so no word appears after it).
After “*in*”, only “*the*” appears once.
“*hat*” appears only at the end of document 2, so we do not emit it as a key.

Write map, combine, and reduce functions to solve this task. Pseudo-code is acceptable, but it should still be precisely clear what your program is doing at each step. You can assume the existence of a string splitting function of your choice, e.g. `split()`. Your program should be able to scale over multiple machines, and your combine function should improve its efficiency. Follow the function formats below.
[13 marks]

```
map (document_id, line) {
  /* your pseudo code*/
  /* you need to output the map results by calling the API,
  emit(specify your map output)*/
}

combine (/* specify your input to combiner */) {
  /* your pseudo code */
  /* emit appropriate outputs for the combiner to work */
}
```

```

reduce (/* specify your input to reducer */) {
/* your pseudo code */
/* emit appropriate outputs for the reducer to work */
}

```

Your Answer:

```

map (document_id, line) {
    words = split(line)
    for i in range(len(words) - 1):
        emit(words[i], (words[i+1], 1))
}

combine (key, w_counts){

    // combine and emit local counts
    counts = Counter()
    for (w, c) in w_counts:
        counts[w] += c
    for w in counts:
        emit(key, (w, counts[w]))
}

reduce (key, w_counts){

    // compute total counts
    counts = Counter()
    for (w, c) in w_counts:
        counts[w] += c

    // find all words with highest frequency
    max_count = max(counts.values())
    max_w = [w for w in counts if counts[w] == max_count]
    emit(key, max_w)
}

```

Explanation: For each key, the combine() function locally aggregates counts for each next word. The reduce() function first finds the max frequency value, then finds all words with this frequency value.

Grading scheme: 9 marks for map/reduce, 4 marks for combine.

QUESTION 3: Data Centers [6 marks]

Your company is using a data center to serve a distributed MongoDB database, called **products**. The data read latency and bandwidth of different hardware components are given in Table 1. Assume that MongoDB operations access disk directly (ignore caching), and assume that documents are evenly distributed across racks.

Table 1. $1\text{ ns} = 1e-9\text{ second}$, $1\text{ }\mu\text{s} = 1e-6\text{ second}$, $1\text{ ms} = 0.001\text{ second}$.

	Latency	Bandwidth
Main memory (DRAM)	100 ns	40 GB/s
Hard disk	10 ms	200 MB/s
Rack switch	300 μs	200 MB/s
Cluster switch	500 μs	10 MB/s

(1) You run the following MongoDB query, which inserts a single document to the database:

```
db.products.insert({product_id: "123", name="apple"})
```

The query takes about 10ms to execute, but your CEO feels that this is too slow and wants to speed it up. Which of the following improvements to the system is likely to most significantly improve the performance of this query? Explain your answer.

- (i) Reduce the latency of the hard disks
- (ii) Increase the bandwidth of the hard disks
- (iii) Reduce the latency of the rack and cluster switches
- (iv) Increase the bandwidth of the rack and cluster switches

[3 marks]

Your Answer:

The query involves transferring a small amount of data over the cluster, so the latency matters most. Here the latency is bottlenecked by the latency of disk transfers (10ms). Hence, this can only be reduced by (i).

(2) You run the following MongoDB query, which reads all documents from the database. Query results are combined and returned to a single client node.

```
db.products.find({})
```

The query takes about 1 hour to execute, and this time consumption is not due to the processing speed of any individual machine. Your CEO feels that this is too slow and wants to speed it up. Which of the following improvements to the system is likely to most significantly improve the performance of this query? Explain your answer.

- (i) Reduce the latency of the hard disks
- (ii) Increase the bandwidth of the hard disks
- (iii) Reduce the latency of the rack and cluster switches
- (iv) Increase the bandwidth of the rack and cluster switches

[3 marks]

Your Answer:

Since we are transferring a large amount of data, the bandwidth matters most. Here the bottleneck is the low bandwidth of 10MB/s of the cluster switch, since it is much lower than the bandwidth of disk transfers, so this bandwidth can only be improved by (iv).

QUESTION 4: Spark [6 marks]

Chris is working for an online electronic devices shop. On a spark cluster, Chris creates a DataFrame table **df** for testing. **df** only includes a few customer transaction records as shown below.

df

customer_id	name	signup_date	age	loyalty_member	country	product	quantity	purchase_amount
1	Alice	2022-01-15	29	true	Singapore	Laptop	2	200.0
2	Bob	2021-11-23	35	false	USA	Smartphone	1	150.0
3	Charlie	2023-02-10	41	true	Singapore	Laptop	1	300.0
4	David	2022-07-08	24	false	India	Tablet	3	100.0
5	Eve	2023-05-14	37	true	USA	Laptop	1	250.0
6	Frank	2021-09-30	32	false	India	Smartphone	2	180.0

Chris did a test run on this table using the below Spark code:

```
1 df_loyalty = df.filter(col("loyalty_member") == True)
2 df_purchase= df_loyalty.groupBy("country").agg(sum("purchase_amount").alias("total_purchase"))
3 df_purchase.show()
4 df_quantity= df_loyalty.groupBy("country").agg(sum("quantity").alias("total_quantity"))
5 df_quantity.show()
6 df_final = df_purchase.join(df_quantity, on="country")
7 df_final.show()
```

- 1) Provide the DataFrame table **df_final** (with header and values) Chris will get by running the above code.

[2 mark]

Your Answer:

country	total_purchase	total_quantity
Singapore	500.0	3
USA	250.0	1

Deduct 0.5 mark for each error (the order of record does not matter)

- 2) After the test run, Chris decided to run the code using the past ten years customer transaction records. At which line(s) of the code, Chris is likely to encounter the potential performance bottlenecks? List the line number(s) and provide the explanation.

[2 mark]

Your Answer:

Line 2 and 4 : if the dataframe `df_loyalty` is not partitioned by country, group all the records by country will require network shuffling (1 mark).

if the grouped data is highly skewed (i.e. some country has more records and some country has very few records). Aggregation tasks may have task straggler issue, i.e. certain task takes much longer time than the other tasks to complete. (1 mark)

Note: line 6 is joining two aggregated tables (with max of 200+ records). The data fits easily in memory and thus the join is done in-memory, with no significant overhead and expensive shuffling required.

- 3) Chris reviewed the code again and think he can simplify the code. Please provide your suggestion to Chris on how to simplify the code. Provide the line number(s), the simplified code and the explanations.

[2 mark]

Your Answer:

We can simplify the line 2- 6 to below code:

```
df_loyalty = df.filter(col("loyalty_member") == True)
df_final=
df_loyalty.groupby("country").agg(sum("purchase_amount").alias("total_purchase"), sum("quantity").alias("total_quantity"))
```

In this way, we combine the two groupby into one groupby and then no join is required.

(1 mark for correct code, and 1 mark for correct explanation)

QUESTION 5: Streaming [6 marks]

1) You are working for an online bank, and you would like to track customers credit card transactions in real-time and calculate the total spend for different locations. You use Spark Streaming to implement this:

```
1 (transactions
2   .withWatermark("timestamp", "2 minutes")
3   .groupBy("location", window("timestamp", "10 minutes", "5 minutes"))
4   .agg(sum("amount").alias("total_spend")))
```

Assume from 9:05 to 9:10, the below transactions are received at Spark cluster under **transactions** stream. Here, timestamp represents the exact time the credit card is used at the merchandise.

transaction_id	user_id	amount	currency	merchant	location	timestamp
TXN30001	U12345	100	USD	Amazon US	New York	10/15/2025 9:01
TXN30002	U67890	70	USD	Starbucks	New York	10/15/2025 9:06
TXN30003	U13579	300	USD	Apple Store	San Francisco	10/15/2025 9:06
TXN30004	U24680	20	USD	7-Eleven	San Francisco	10/15/2025 9:07
TXN30005	U98765	500	USD	Walmart	Chicago	10/15/2025 9:08

At 9:10, the below result table is triggered at Spark cluster:

time_start	time_end	location	total spend
10/15/2025 8:55	10/15/2025 9:05	New York	100
10/15/2025 8:55	10/15/2025 9:05	San Francisco	0
10/15/2025 8:55	10/15/2025 9:05	Chicago	0
10/15/2025 9:00	10/15/2025 9:10	New York	170
10/15/2025 9:00	10/15/2025 9:10	San Francisco	320
10/15/2025 9:00	10/15/2025 9:10	Chicago	500
10/15/2025 9:05	10/15/2025 9:15	New York	70
10/15/2025 9:05	10/15/2025 9:15	San Francisco	320
10/15/2025 9:05	10/15/2025 9:15	Chicago	500

From 9:10 to 9:15, a few more transactions are received at Spark cluster as shown below:

transaction_id	user_id	amount	currency	merchant	location	timestamp
TXN30006	U54321	70	USD	eBay	San Francisco	10/15/2025 9:03
TXN30007	U45678	250	USD	Best Buy	Chicago	10/15/2025 9:04
TXN30009	U86420	50	USD	Starbucks	San Francisco	10/15/2025 9:08
TXN30008	U33333	100	USD	Walmart	New York	10/15/2025 9:11
TXN30010	U75319	30	USD	7-Eleven	New York	10/15/2025 9:13

Please provide the result table triggered at Spark cluster at 9:15.

(**Note:** DO NOT include in this table the records that are already dropped from Spark intermediate state)

[4 marks]

Your Answer:

time_start	time_end	location	total spend
10/15/2025 9:00	10/15/2025 9:10	New York	170
10/15/2025 9:00	10/15/2025 9:10	San Francisco	440
10/15/2025 9:00	10/15/2025 9:10	Chicago	750
10/15/2025 9:05	10/15/2025 9:15	New York	200
10/15/2025 9:05	10/15/2025 9:15	San Francisco	370
10/15/2025 9:05	10/15/2025 9:15	Chicago	500
10/15/2025 9:10	10/15/2025 9:20	New York	130
10/15/2025 9:10	10/15/2025 9:20	San Francisco	0
10/15/2025 9:10	10/15/2025 9:20	Chicago	0

During the processing time period: 9:10-9:15, $w_m = 9:08 - 2\text{min} = 9:06$

Thus, the window (8:55-9:05) is dropped.

Deduct 1 mark for each error (if student makes 4 errors or more then zero mark)

The order of the records can be different. If student has missing or extra record, then deduct 1 mark for each missing or extra record.

2) You heard Flink can provide more real-time streaming processing. Therefore, you used Flink to process the same transactions and also injected several Flink watermark records as required. All the transactions are summarized in the below table. Here, “timestamp” represents the exact time the credit card is used at the merchandise. “received_at_Flink” represents the time the record is received at Flink.

transaction_id	user_id	amount	currency	merchant	location	timestamp	received_at_Flink
TXN30001	U12345	100	USD	Amazon US	New York	10/15/2025 9:01	10/15/2025 9:02
Watermark #1						10/15/2025 9:00	10/15/2025 9:02
TXN30002	U67890	70	USD	Starbucks	New York	10/15/2025 9:06	10/15/2025 9:07
TXN30003	U13579	300	USD	Apple Store	San Francisco	10/15/2025 9:06	10/15/2025 9:08
TXN30004	U24680	20	USD	7-Eleven	San Francisco	10/15/2025 9:07	10/15/2025 9:09
TXN30005	U98765	500	USD	Walmart	Chicago	10/15/2025 9:08	10/15/2025 9:10
Watermark #2						10/15/2025 9:05	10/15/2025 9:10
TXN30006	U54321	70	USD	eBay	San Francisco	10/15/2025 9:03	10/15/2025 9:11
TXN30007	U45678	250	USD	Best Buy	Chicago	10/15/2025 9:04	10/15/2025 9:11
TXN30009	U86420	50	USD	Starbucks	San Francisco	10/15/2025 9:08	10/15/2025 9:11
Watermark #3						10/15/2025 9:10	10/15/2025 9:12
TXN30008	U33333	100	USD	Walmart	New York	10/15/2025 9:11	10/15/2025 9:15
TXN30010	U75319	30	USD	7-Eleven	New York	10/15/2025 9:13	10/15/2025 9:15

Please provide at what time Flink is likely to trigger the **on-time** results for event time window 10/15/2025 8:55 to 10/15/2025 9:05.

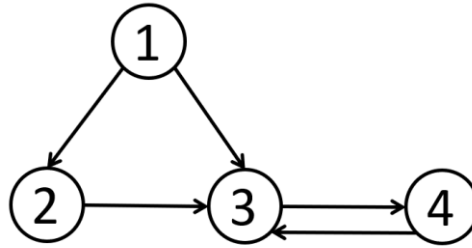
[2 marks]

Your Answer:

At 10/15/2025 9:10 (after received Watermark#2 with event time 9:05).

QUESTION 6: Graphs [7 marks]

Joel is analysing a small network with four webpages. The relationships between these four webpages are shown in the below graph:



As you can see, webpage 1 has hyperlinks pointing to webpages 2 and 3, webpage 2 has hyperlink pointing to webpage 3, webpage 3 has hyperlink pointing to webpage 4 and webpage 4 has hyperlink pointing to webpage 3. Among all the four webpages, only webpages 3 and 4 are related to Big Data topic.

Joel plans to use Topic Specific PageRank algorithm with teleport ($\beta=0.8$, i.e. teleport probability = $1 - \beta$) to calculate the importance rank of each webpage related to Big Data topic. If a web page is cited by more webpages and more important webpages then it has higher importance rank. We also give the webpages relevant to Big Data topic the priority by only teleporting to these web pages.

Please help Joel to solve this problem step by step.

- 1) Joel used matrix form to solve this problem:

$$\mathbf{r}_{t+1} = \mathbf{A} \cdot \mathbf{r}_t$$

Here, $\mathbf{r}_t$ and $\mathbf{r}_{t+1}$ represent the importance vector for all the webpages at iteration t and $t+1$ respectively. $\mathbf{A}$ represents the Matrix for Topic Specific PageRank algorithm.

$$A_{ij} = \begin{cases} \beta M_{ij} + (1 - \beta)/|S| & \text{if } i \in S \\ \beta M_{ij} + 0 & \text{otherwise} \end{cases}$$

Here, S represents the topic specific set relevant to Big Data topic.

Please provide the Matrix $\mathbf{A}$ for Joel. (Round the final numbers in Matrix $\mathbf{A}$ to two decimal points)

[2 marks]

Your Answer:

$$M = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, N = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}$$

$$A = \beta M + (1 - \beta)N = 0.8 M + 0.2N = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.4 & 0 & 0 & 0 \\ 0.5 & 0.9 & 0.1 & 0.9 \\ 0.1 & 0.1 & 0.9 & 0.1 \end{bmatrix}$$

(0.5 mark for each column of matrix A)

2) Assume Joel initialized the importance vector $\mathbf{r}_0$ as:

$$\mathbf{r}_0 = \begin{bmatrix} 0.25 \\ 0.25 \\ 0.25 \\ 0.25 \end{bmatrix}$$

Then, he ran the topic sensitive page rank algorithm for **three** iterations.

Please calculate the importance vector $\mathbf{r}_t$ for iteration 1 to 3 and provide the results. (Round to two decimal places)

[3 marks]

Your Answer:

Iteration 1 to 3

$$\mathbf{r}_1 = \begin{bmatrix} 0 \\ 0.10 \\ 0.60 \\ 0.30 \end{bmatrix} \quad \mathbf{r}_2 = \begin{bmatrix} 0 \\ 0 \\ 0.42 \\ 0.58 \end{bmatrix} \quad \mathbf{r}_3 = \begin{bmatrix} 0 \\ 0 \\ 0.56 \\ 0.44 \end{bmatrix}$$

(1 mark for each iteration)

- 3) Based on the above results, do you observe any problem of applying topic specific page rank algorithm on this small network? Please provide your observation and recommendation for Joel to fix this problem.

[2 mark]

Your Answer:

Topic specific page rank algorithm only teleport to the topic specific set. However, in this small network, the topic specific set (webpages 3 and 4) is a spider trap. Therefore, all the importance values are trapped here in webpages 3 and 4. (1 mark)

To avoid all the importance values trapped in webpages 3 and 4, we can adjust the topic specific rank algorithm by allowing the random walker to teleport to webpages 1 and 2 with a small probability (instead of zero) compared to webpages 3 and 4. (1 mark)

Below is optional, just to help the grader to understand the above point.

For example, we can adjust matrix N from N_{old} to N_{new}.

$$N_{old} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix} \quad N_{new} = \begin{bmatrix} 0.05 & 0.05 & 0.05 & 0.05 \\ 0.05 & 0.05 & 0.05 & 0.05 \\ 0.45 & 0.45 & 0.45 & 0.45 \\ 0.45 & 0.45 & 0.45 & 0.45 \end{bmatrix}$$

BLANK PAGE
END OF PAPER